56262

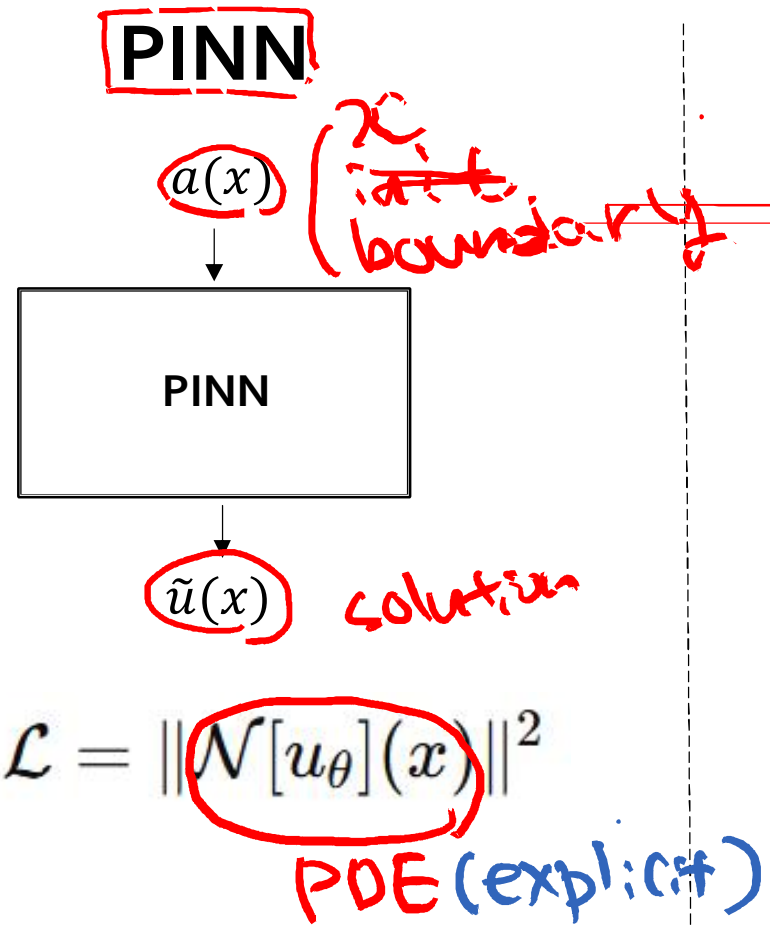
Neural Operator 1

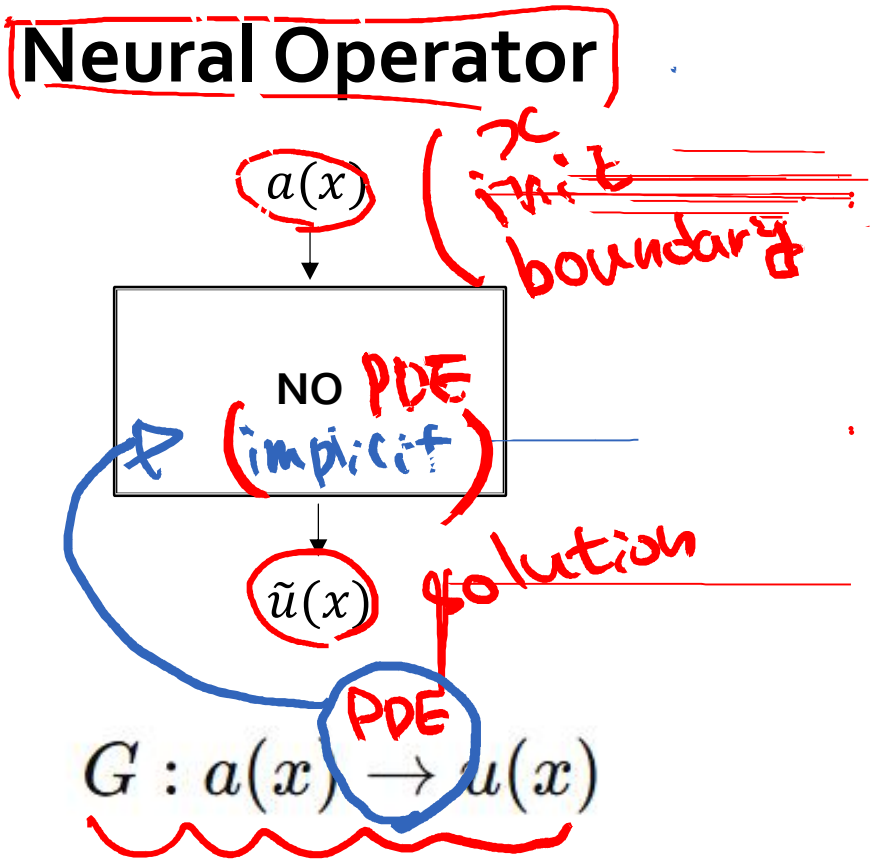
Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs
(*Journal of Machine Learning Research, 2021*)

Problem of Vanilla PINN

Problem of Vanilla PINN	Alternatives	Keywords
Instance-wise retraining	Neural Operator	Operator learning
Noisy environments	Bayesian PINN	Uncertainty
Loss/gradient imbalance	Self-adaptive PINN	Self-adaptive weighting
Unstable training	X-PINN	Domain decomposition

PINN vs. Neural Operator 1/3





- Minimizes PDE residual
- Maps input function to solution function

PINN vs. Neural Operator 2/3

	PINN	Neural Operator
Loss Weighting	Solve a specific PDE	Learn a PDE solver
Learning	Physics-based loss	Data-driven
Generalization	Limited	Strong
Speed	Slow (train per problem)	Fast (once trained)
Data requirement	Low	High

- **Neural Operator**
  - Learns a mapping between functions (operator learning)
  - Approximates the solution operator of a PDE

PINN vs. Neural Operator 3/3

- **When to Use**

PINN	Neural Operator
Insufficient data available	Sufficient training data
Physics is well-defined	Fast inference is important (Thousands of times faster)
A single accurate solution	Solve many similar PDEs
Inverse problems, Parameter estimation	CFD, Weather prediction (Various Initial and Boundary conditions)

Problem Statement 1/3

- **Neural operators :** operators (mapping) between infinite-dimensional function spaces
  - Discretized by a variety of different methods
  - At different levels of resolution
  - Without the need for re-training.

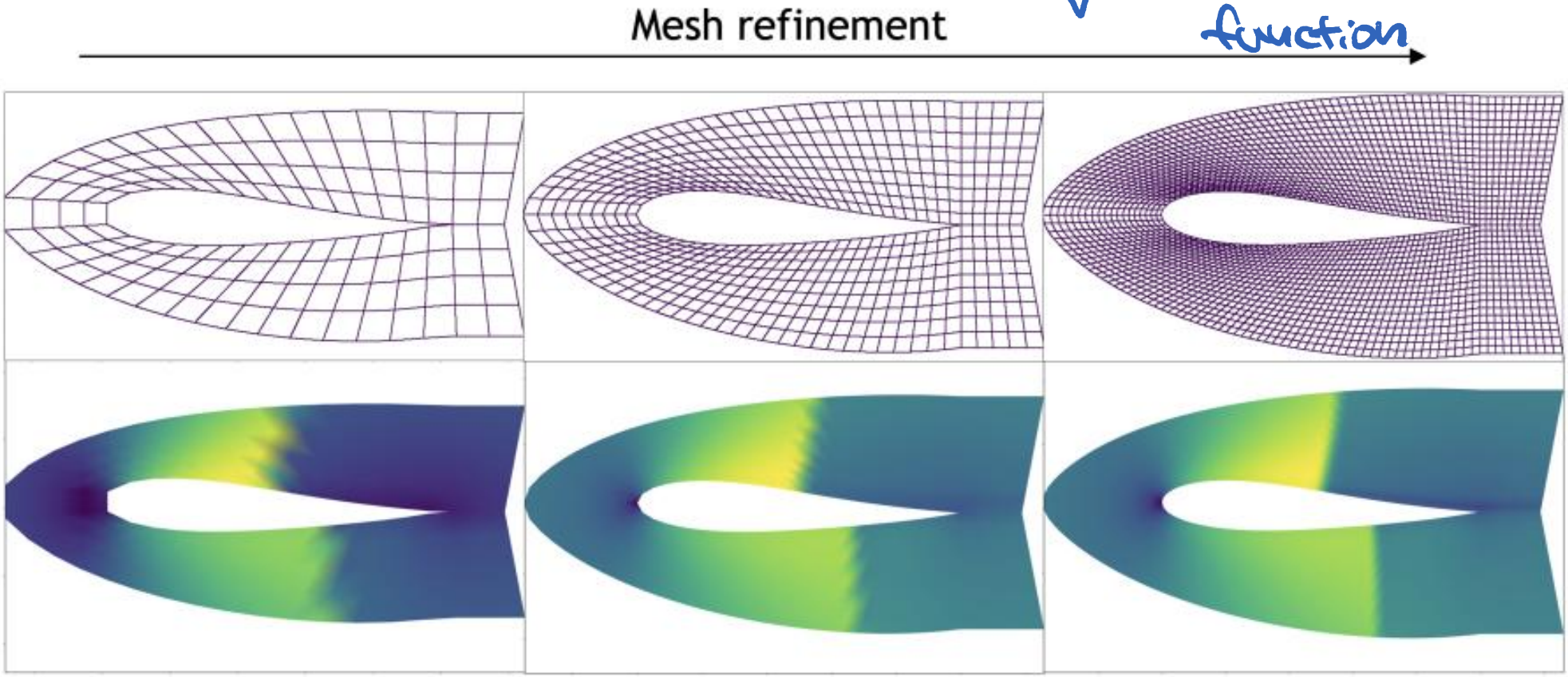
- **Discretization Invariance**
  - **(Input) any grid :** It can take inputs defined on any set of points (vs. CNN)
  - **(Output) any grid :** It can produce outputs at any point
  - (lim Its predictions converge to a continuous operator as the discretization becomes finer (vs. GNN, Transformer)

Problem Statement 2/3

- **Discretization Invariance**

lim $\Delta x \to 0$

⇩ Continuous function



Problem Statement 3/3

- **Key Advantages**
  - Works on any resolution / mesh
  - Same parameters across discretizations
  - Consistent in function space
  - Naturally suited for PDE problems
- **Implementation Variants (Kernel)**
  - Graph Neural Operator (GNO)
  - Low-Rank Neural Operator (LNO)
  - Multipole Neural Operator
  - Fourier Neural Operator (FNO)

Generic Parametric PDEs

- **Key Idea**
  - A PDE can be viewed as a mapping from an input function to a solution function

$$(L_a u)(x) = f(x), \quad x \in D$$
$$u(x) = 0, \quad x \in \partial D$$

Physics

initial + boundary

ex) Poisson equation

$$-\nabla \cdot (a(x)\nabla u(x)) = f(x)$$

$$(L_a u)(x) = -\nabla \cdot (a(x)\nabla u(x))$$

- $a \in A$: parameter (input function)
- $u \in U$: solution (output function)
- L_a: operator depending on a

Neural Operator

- **Key Operator Definition**

$$L^{-1}(L_a u)(x) = f(x), \quad x \in D$$

$$G^\dagger := L_a^{-1} f : A \to U$$

- Solving a PDE = applying an operator $G^\dagger$
- Instead of solving the PDE repeatedly, NO Learn the operator that maps input functions to solutions: $a \mapsto u$

Problem Setting 1/2

- **Goal**
  - Learn a mapping between two infinite dimensional spaces by using a finite collection of observations of input-output pairs from this mapping.
  - Learn an approximation of the true operator $G^\dagger$

NN_θ

- **Goal**
  - Learn a mapping between two infinite dimensional spaces by using a finite collection of observations of input-output pairs from this mapping.
  - Learn an approximation of the true operator $G^\dagger$

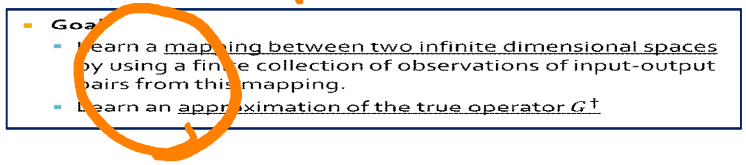
- **Data**

$$\{(a^{(i)}, u^{(i)})\}_{i=1}^{N}$$

- $a^{(i)} \sim \mu$: input functions (sampled from a distribution)
- $u^{(i)} = G^\dagger(a^{(i)})$: corresponding outputs

Problem Setting 1/2

- **Goal**
  - Learn a mapping between two infinite dimensional spaces by using a finite collection of observations of input-output pairs from this mapping.
  - Learn an approximation of the true operator $G^\dagger$



Problem Setting 2/2

- **Data**

$$\{(a^{(i)}, u^{(i)})\}_{i=1}^{N}$$

  - $a^{(i)} \sim \mu$: input functions (sampled from a distribution)
  - $u^{(i)} = G^{\dagger}(a^{(i)})$: corresponding outputs

- **Loss Function**
  - Expected error : $\|G^{\dagger} - G_{\theta}\|_{L^2_{\mu}}^2 = \mathbb{E}_{a\sim\mu}\|G^{\dagger}(a) - G_{\theta}(a)\|_U^2$
  - Empirical-risk minimization: $\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \|u^{(i)} - G_{\theta}(a^{(i)})\|_U^2$

Formulation 1/2

- **Overall Idea**
  - A Neural Operator is a neural network that learns a mapping between functions:

$$G_\theta : a(x) \rightarrow u(x)$$

  - All operations are defined in function space (not on grids)

Formulation 2/2

$$G_\theta : a(x) \to u(x)$$

Input

- **Architecture Overview**
  1. Lifting $a(x) \to v_0(x)$ embedding
     - A pointwise transformation
     - Maps input to a **higher-dimensional feature space**
  2. Iterative Kernel Integration $v_t \to v_{t+1}$

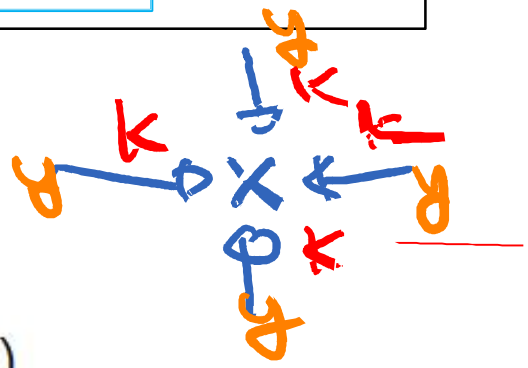
$$v_{t+1}(x) = \sigma\left(W v_t(x) + \int \kappa(x, y) v_t(y) dy + b(x)\right)$$

kernel

  3. Projection $v_T(x) \to u(x)$
     - maps features back to output function

Iterative Kernel Integration

$$v_{t+1}(x) = \sigma\left(W v_t(x) + \int \kappa(x,y) v_t(y) dy + b(x)\right)$$



- **Architecture Overview**
  - Local linear term $W v_t(x)$
    - like standard neural network (matrix)
  - Non-local integral operator $\int \kappa(x,y) v_t(y) dy$
    - information from all points y
    - influences point x
  - Bias + activation $b(x)$
    - standard neural network components

Full Model 1/2

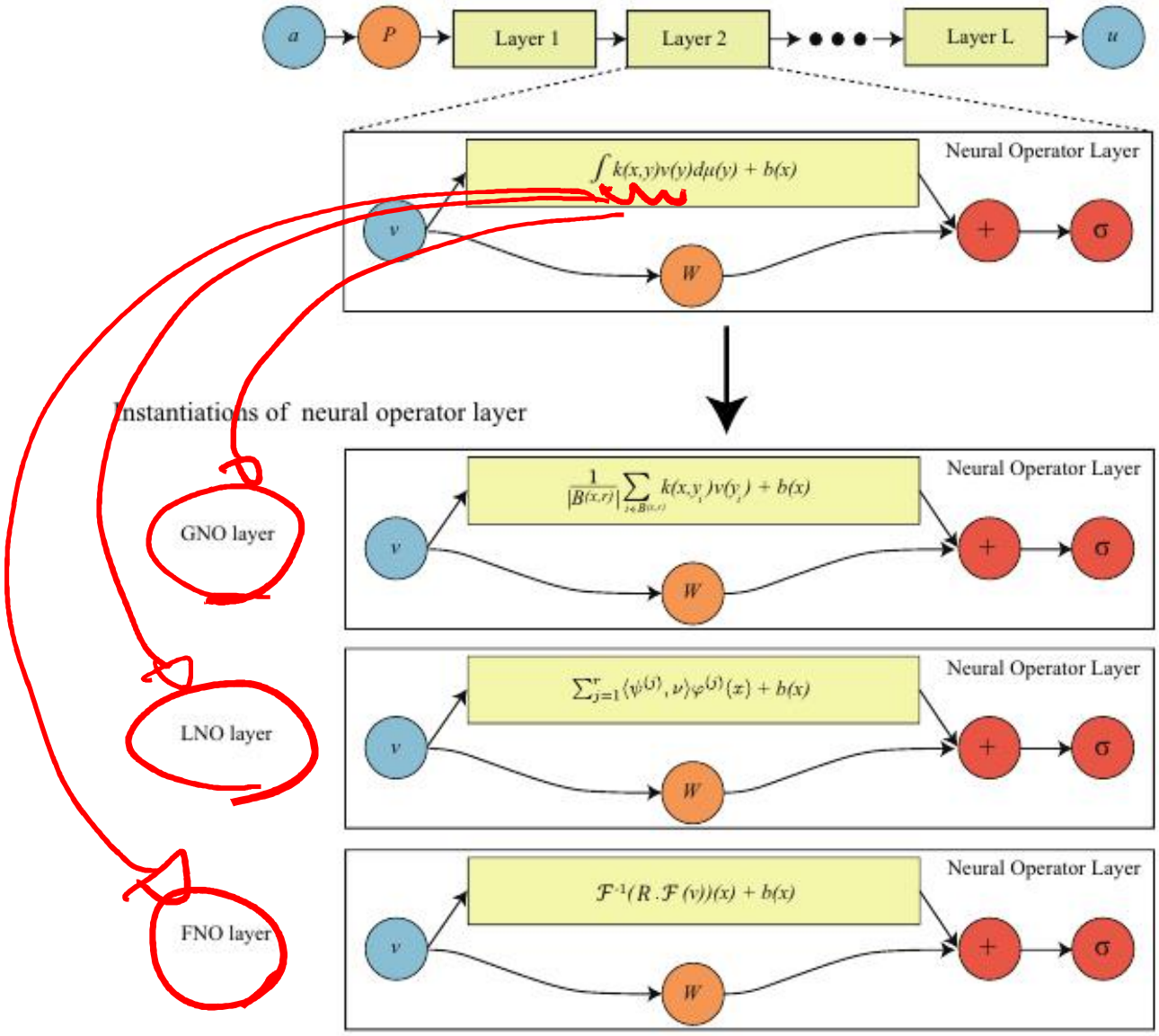
- **Composition**

$$G_\theta = Q \circ \sigma(W + K + b) \circ \cdots \circ P$$

  - layers stacked
  - but layers are **operators, not matrices**

- **Key Difference**
  - Replaces matrix multiplication with integral operators (infinite-dimensional linear operator)

Full Model 2/2



Parameterization 1/3

- **Core Problem**
  - The key operation in Neural Operators is:

$$u(x) = \int_D \kappa(x, y)\, v(y)\, dy$$

    - To compute $u(x)$, we must aggregate information from **all points** y
    - This creates **global interactions**

- **Discretization**
  - Reduce the cost of computing the kernel integral while preserving accuracy

$$u(x_j) \approx \sum_{l=1}^{J} \kappa(x_j, x_l) v(x_l)$$

Parameterization 2/3

- Neural Operator (**Riemann Sum**)
  - The continuous operator in Neural Operators is:

$$u(x) = \int_D \kappa(x, y)\, v(y)\, dy$$

  - In practice, we approximate this integral using a Riemann sum:

$$u(x_j) \approx \sum_{l=1}^{J} \kappa(x_j, x_l)\, v(x_l)\, \Delta x_l$$

Riemann sum

$J \to \infty$
$\Delta x \to 0$
↓
Integral

As the discretization becomes finer ($J \to \infty$):
It **converges to a continuous integral operator**
(Riemann sum → true integral)

Parameterization 3/3

- CNN Convolution (**Discrete Sum**)
  - CNN convolution is:

$$u(x_j) = \sum_{l \in \text{local}} \kappa(x_j - x_l)\, v(x_l)$$

  - It is not treated as a Riemann approximation of a fixed continuous operator (No meaning : $I \to \infty$)

No guarantee of convergence
to a meaningful continuous operator